

Colour-Based Object Tracking and Guidance for ROV Navigation

Emre Yoleri

Technical Report · October 2026

Abstract. Remotely operated vehicles (ROVs) often have to approach a known, brightly coloured target such as a mooring buoy or a docking marker. This report presents a lightweight vision module in Python and OpenCV that segments the target in HSV colour space, cleans the mask with morphological filtering, localizes the target with image moments and converts its offset from the optical axis into yaw, heave and surge commands. Because annotated ROV footage is rarely public, the module was evaluated on a synthetic underwater video generator with exact per-frame ground truth, covering six scenarios (clear water, turbidity, sensor noise, distractors, occlusion and all effects combined). The visible target was detected in 100 % of frames in every scenario, the full command matched the ground-truth command in 95.7–96.4 % of frames, and the tracker ran at 258–531 frames per second on a single CPU core. An ablation study and a saturation-threshold sweep identify which components and parameters matter.

Keywords: colour segmentation, HSV, object tracking, mathematical morphology, ROV guidance.

1. Introduction

Many underwater robotics tasks reduce to driving a vehicle towards a known, colour-coded marker: a buoy, a docking station, a pipeline tag or a competition gate. Such markers are deliberately chosen to stand out from the blue–green background, which makes colour a strong and cheap cue for tracking [1]. Underwater imaging, however, degrades colour in several ways. Water absorbs red light first, so warm colours fade and shift with distance [2]; suspended particles scatter light and add a haze that lowers saturation and contrast [3]; and sediment, fish and uneven lighting introduce colour noise and false candidates.

Learned detectors can handle these effects but need annotated data and a GPU. For small vehicles with an embedded CPU, an interpretable classical pipeline that can be tuned at the dive site is often preferable. The objective of this work is to (i) build a real-time colour-based tracker with OpenCV [4], (ii) turn the target position into simple guidance commands in the spirit of image-based visual servoing [5], and (iii) evaluate localization and command accuracy quantitatively under controlled degradations, including an ablation of each pipeline component.

2. Methodology

2.1. Tracking Pipeline

Each frame passes through the stages shown in Figure 1. A 5×5 Gaussian blur first suppresses sensor noise. The frame is then converted from BGR to HSV [6], which separates hue (*what* colour) from value (*how bright*), so that the mask is less sensitive to uneven lighting. Pixels between the lower bound $(H, S, V) = (5, 80, 60)$ and the upper bound $(22, 255, 255)$ (orange preset) are kept; a wrap-around range with $H_{\text{low}} > H_{\text{high}}$ handles red, which straddles $H = 0$. The binary mask is cleaned with a morphological opening (5×5 ellipse), which removes isolated specks, followed by a closing (9×9 ellipse), which fills holes left by specular highlights and shadows [7].

External contours are extracted by border following [8], and contours with an area below 150 px are rejected as small same-coloured objects. The largest remaining contour is taken as the target, and its centroid is computed from image moments [9] as $(c_x, c_y) = (m_{10}/m_{00}, m_{01}/m_{00})$. A temporal filter then applies an exponential moving average (EMA) with weight $\alpha = 0.5$ to (c_x, c_y, A) . When detection fails, the last estimate is held for up to 10 frames (coasting) before the state switches to searching.

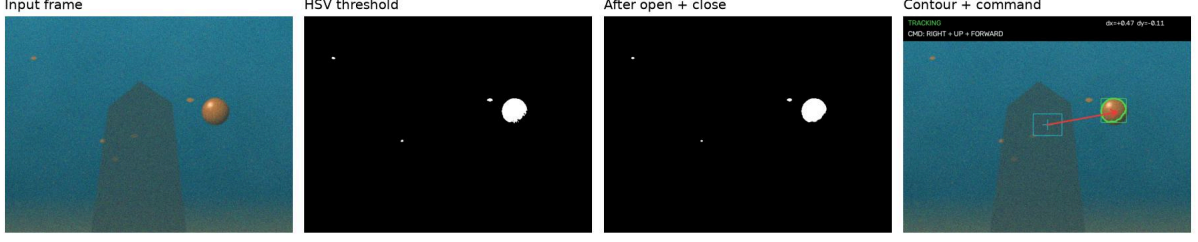


Figure 1. Pipeline stages on a frame from the hardest scenario: input, HSV threshold, mask after opening and closing, and selected contour with the resulting command.

2.2. Guidance Law

For a frame of width W and height H , the target offset is normalized to $[-1, 1]$:

$$d_x = \frac{c_x - W/2}{W/2}, \quad d_y = \frac{c_y - H/2}{H/2}. \quad (1)$$

Yaw is CENTER if $|d_x| < \delta$ and LEFT or RIGHT otherwise; heave is LEVEL if $|d_y| < \delta$ and UP or DOWN otherwise, with a dead zone $\delta = 0.1$ that prevents thruster chatter near the centre. Surge is FORWARD until the target covers 8 % of the frame and STOP afterwards, since the apparent area of a sphere grows with the inverse square of its distance. A lost target produces a SEARCH command. Proportional rates (d_x , d_y , zero inside the dead zone) are also output for a P-controller.

2.3. Synthetic Evaluation Data

Real ROV footage with frame-accurate target positions is rarely public. A synthetic generator therefore renders a shaded orange buoy that follows a known Lissajous path and grows as the vehicle approaches, which gives exact ground truth for every frame. The water model includes a depth gradient, seabed and backscatter blotches, Gaussian sensor noise, brightness flicker and defocus blur. Six scenarios were built on top of it (Figure 2): *clear*; *turbid* (blending towards the water colour, which desaturates the buoy); *noisy* (400–500 reddish 1–2 px sediment specks per frame); *distractors* (small orange fish); *occlusion* (the buoy passes behind a rock; frames with less than 25 % of the buoy visible count as not visible); and *hard*, which combines all effects. Each scenario has 300 frames of 640×480 pixels and was run with two random seeds.

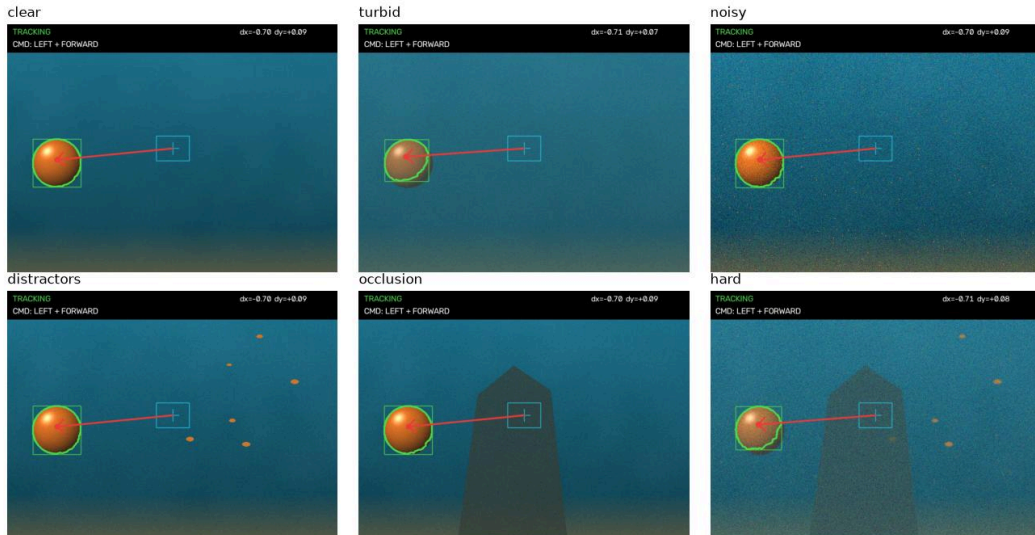


Figure 2. The six synthetic test scenarios with the tracker overlay (cyan box: dead zone; red arrow: correction vector).

2.4. Evaluation Metrics

The *detection rate* is the share of visible frames in which the detected centroid lies inside the true buoy radius. *False-positive (FP) frames* are frames with a detection that is either off target or made while the

buoy is not visible. The *centroid error* is the Euclidean distance between the detected and true centres, and the *mask IoU* is the overlap of the cleaned mask with the true visible buoy disk. *Command accuracy* is the share of visible frames in which the yaw, heave and surge commands (and all three together, “full”) equal those computed from the ground truth. *Spurious blobs* counts extra connected components in the cleaned mask, and *command changes* per 100 frames measures chatter. Throughput (FPS) covers the tracker only, on one CPU core of an Apple Silicon laptop.

3. Results

3.1. Tracking and Command Accuracy

Table 1 summarizes the results. The visible target was found in every frame of every scenario, and the full command matched the ground truth in 95.7–96.4 % of frames, with per-axis accuracy between 97.0 % and 98.9 %. Almost all mismatches occur when the target crosses the edge of the dead zone: the EMA lags the true position by one or two frames, so the command switches slightly late. The tracker needs 2–4 ms per frame, far above the 25–30 FPS of a typical ROV camera; the spread in FPS is due to other processes running during the benchmark.

Table 1. Results per scenario (300 frames $\times$ 2 seeds, 640 $\times$ 480). Detection, FP frames and command accuracy (yaw, heave, surge, full) in %, computed over visible frames.

Scenario	Det.	FP	Err. (px)	IoU	Yaw	Heave	Surge	Full	FPS
Clear	100	0.00	0.96	0.94	98.33	97.33	100	95.67	258
Turbid	100	0.00	5.48	0.72	98.67	97.00	100	95.67	531
Noisy	100	0.00	1.45	0.92	98.33	97.67	100	96.00	286
Distractors	100	0.00	1.05	0.76	98.33	97.50	100	95.83	428
Occlusion	100	6.33	1.28	0.93	98.93	97.51	100	96.44	479
Hard	100	1.33	4.65	0.69	98.93	97.15	100	96.09	492

Turbidity is the hardest effect. Haze pushes the saturation of the shadowed half of the buoy below the threshold, so the mask mainly covers the lit half (IoU 0.72) and the centroid is biased by about 5 px towards the light. The command remains correct because this bias is small compared with the 32 px dead zone. In the occlusion scenario, the 6.33 % FP frames are frames in which a thin sliver of the buoy (less than 25 % visible) is still detected: technically correct detections that the strict protocol counts as errors. Figure 3 shows that the smoothed trajectory follows the ground truth closely; during the occlusion the tracker coasts on its last estimate and recovers as soon as the buoy reappears.

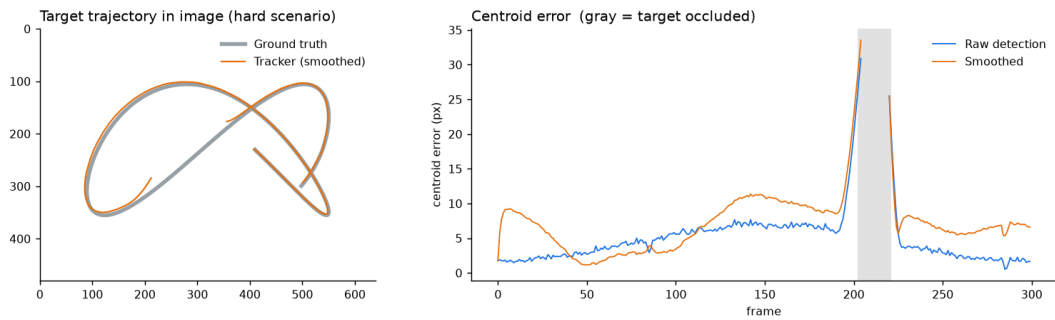


Figure 3. Hard scenario: tracked vs. true trajectory (left) and centroid error over time (right; grey band = target occluded).

3.2. Ablation Study

Each component was removed in turn (Table 2). Without morphological filtering, the *noisy* scenario leaves 25.42 spurious blobs per frame instead of 0.01, and the mask IoU drops from 0.92 to 0.87. The largest-contour rule still selects the buoy, but any step that uses the whole mask would be flooded with noise. Without the minimum-area filter, FP frames in the *hard* scenario rise from 1.33 % to 6.33 % and command changes from 3.68 to 5.52 per 100 frames, because the tracker locks onto an orange fish while the buoy is hidden. Removing temporal smoothing, in contrast, *raises* the full command accuracy by

1.5–3 percentage points (e.g. 95.67 % $\rightarrow$ 98.67 % in *clear*) and cuts the centroid error of the reported position from 5.60 px to 0.96 px, with no change in chatter.

Table 2. Ablation of pipeline components on the *noisy* and *hard* scenarios.

Configuration	Scenario	FP (%)	IoU	Blobs/frame	Full (%)
Full pipeline	Noisy	0.00	0.92	0.01	96.00
w/o morphology	Noisy	0.00	0.87	25.42	96.00
w/o smoothing	Noisy	0.00	0.92	0.01	98.33
Full pipeline	Hard	1.33	0.69	3.99	96.09
w/o min-area filter	Hard	6.33	0.69	3.99	95.73
w/o smoothing	Hard	1.33	0.69	3.99	97.86

3.3. Sensitivity to the Saturation Threshold

The lower saturation bound $S_{\min}$ is the most important parameter (Figure 4). In clear water the mask IoU stays at 0.90–0.94 up to $S_{\min} = 140$, but in turbid water the IoU falls from 0.78 at $S_{\min} = 30$ to 0.54 at 110 and the target is lost entirely at 140 (IoU 0.01). An earlier preset with $S_{\min} = 120$ roughly doubled the turbid centroid error (10.7 px vs. 5.5 px), so the default was lowered to 80, which keeps a margin against turbidity while still rejecting desaturated background.

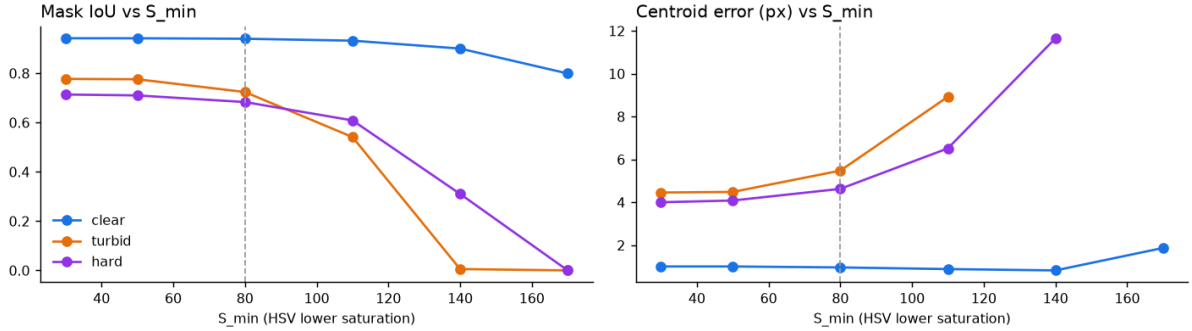


Figure 4. Mask IoU and centroid error as a function of $S_{\min}$ (dashed line: default value 80).

4. Discussion

The results show that a classical colour pipeline is sufficient for reliable, real-time guidance towards a distinctive marker: detection was perfect on all visible frames and the commands were correct in about 96 % of frames, at roughly ten times the frame rate of a typical ROV camera. The ablation shows that each cleaning step targets a specific failure mode: morphology removes colour noise inside the target hue band, and the minimum-area filter prevents lock-on to small distractors during occlusion.

Temporal smoothing was the one component that hurt the metrics. This is an honest negative result of the synthetic setup: the virtual camera is perfectly steady and the segmentation is very stable, so the EMA only adds lag. On a real vehicle, wave-induced pitch and roll and flickering caustics make the raw measurements jitter, which is where smoothing is needed; α is therefore exposed as a tunable parameter ($\alpha = 1$ disables it). The saturation sweep reflects the physics of underwater colour loss: any fixed threshold trades robustness to turbidity against rejection of background.

The main limitations are the following. First, the tracker is colour-only, so any larger object of the same hue wins the largest-contour rule; shape gating with the circularity measure already computed, or a search window around the predicted position, would help. Second, colour constancy is not handled: an orange buoy turns brown and then grey with depth and distance, and underwater white balance with red channel compensation [10] before segmentation would widen the working range. Third, the evaluation uses synthetic video only, and the target area is only a relative proxy for distance.

5. Conclusion

A real-time colour-based tracker for ROV navigation was developed in Python and OpenCV. It combines HSV thresholding, morphological cleaning, contour filtering, moment-based localization and a dead-zone guidance law. On six synthetic underwater scenarios with exact ground truth, it detected the visible target in 100 % of frames, produced the correct full command in 95.7–96.4 % of frames and ran at 258–531 FPS on one CPU core. The method is also available in an interactive Streamlit application with live parameter tuning and HSV auto-calibration. Future work includes a constant-velocity Kalman filter [11] to replace the lagging EMA, metric range estimation from the known buoy diameter, colour correction before segmentation, and validation on real pool or competition footage with hand-labelled target centres.

References

- [1] A. Yilmaz, O. Javed, and M. Shah, “Object Tracking: A Survey,” *ACM Computing Surveys*, vol. 38, no. 4, 2006, doi: 10.1145/1177352.1177355.
- [2] D. Akkaynak and T. Treibitz, “A Revised Underwater Image Formation Model,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6723–6732.
- [3] J. S. Jaffe, “Computer Modeling and the Design of Optimal Underwater Imaging Systems,” *IEEE Journal of Oceanic Engineering*, vol. 15, no. 2, pp. 101–111, 1990, doi: 10.1109/48.50695.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobb's Journal of Software Tools*, 2000.
- [5] F. Chaumette and S. Hutchinson, “Visual Servo Control, Part I. Basic Approaches,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006, doi: 10.1109/MRA.2006.250573.
- [6] A. R. Smith, “Color Gamut Transform Pairs,” *ACM SIGGRAPH Computer Graphics*, vol. 12, no. 3, pp. 12–19, 1978, doi: 10.1145/965139.807361.
- [7] J. Serra, *Image Analysis and Mathematical Morphology*. London: Academic Press, 1982.
- [8] S. Suzuki and K. Abe, “Topological Structural Analysis of Digitized Binary Images by Border Following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985, doi: 10.1016/0734-189X(85)90016-7.
- [9] M.-K. Hu, “Visual Pattern Recognition by Moment Invariants,” *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179–187, 1962, doi: 10.1109/TIT.1962.1057692.
- [10] C. O. Ancuti, C. Ancuti, C. De Vleeschouwer, and P. Bekaert, “Color Balance and Fusion for Underwater Image Enhancement,” *IEEE Transactions on Image Processing*, vol. 27, no. 1, pp. 379–393, 2018, doi: 10.1109/TIP.2017.2759252.
- [11] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.